

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Logic Design with HDL (CO1026)

Assignment Report

Designing a simple RISC Processor

Instructor: Nguyễn Thành Lộc

Students:	Trần Vinh Quang	2550342
	Lê Hiễn Vinh	2453422
	Nguyễn Đức Thiên Tân	2550345
	Đặng Ngọc Lan Anh	2550258

HO CHI MINH CITY, JUNE 2026



Contents

1	Introduction	3
1.1	Project Overview	3
1.2	Objectives	3
1.3	Tools and Environment	4
1.4	Main Features of the CPU	4
1.5	Report Organization	5
2	System Design Approach	5
2.1	Theoretical Overview	5
2.2	Top-level System Architecture	7
2.3	Fetch Phase	7
2.4	Execution Phase	7
2.5	Functional Blocks	8
3	Implementation	12
3.1	Implementation Strategy	12
3.2	Program Counter (PC)	12
3.3	Address Multiplexer (Address MUX)	13
3.4	Arithmetic Logic Unit (ALU)	14
3.5	Controller Unit	16
3.6	Instruction Register (IR)	21
3.7	Accumulator Register (AC)	23
3.8	Memory	24
3.9	Top-level Integration	25
4	Testing Simulation and Validation	29
4.1	Program Counter (PC)	29
4.2	Address Multiplexer (Address MUX)	33
4.3	Arithmetic Logic Unit (ALU)	37
4.4	Controller Unit	42
4.5	Instruction Register (IR)	47
4.6	Accumulator Register (AC)	51
4.7	Memory	55
4.8	Top-level Integration	59
4.9	Resource, Timing, and Power Evaluation	64



5	Conclusion and Future Work	65
5.1	Conclusion	65
5.2	Difficulties Encountered	66
5.3	Future Plan	67

1 Introduction

1.1 Project Overview

A Central Processing Unit (CPU) is the core component of a computer system, responsible for fetching instructions, decoding them, executing operations, and controlling data movement between internal registers and memory. In this project, a simple Reduced Instruction Set Computer (RISC) CPU is designed using *Verilog Hardware Description Language (HDL)*.

The proposed CPU follows a simplified RISC architecture with a compact instruction format. Each instruction consists of a 3-bit opcode and a 5-bit operand field, allowing the processor to support eight basic instructions and address a memory space of 32 locations. Although the architecture is simple, it demonstrates the fundamental principles of processor design, including instruction fetching, instruction decoding, arithmetic and logic execution, memory access, and program control.

The CPU is organized as a set of functional hardware modules, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, Arithmetic Logic Unit, and Controller. These modules are designed independently and then integrated into a complete processor system. The CPU operates based on clock and reset signals and continues executing instructions until a halt instruction is encountered.

1.2 Objectives

The main objective of this project is to design, implement, and verify a simple RISC CPU at the register-transfer level using Verilog HDL. Through this project, the design process of a basic processor can be studied from both theoretical and practical perspectives.

The specific objectives of the project are as follows:

- To understand the fundamental organization and operation of a RISC-based CPU.
- To design the main functional blocks of the CPU, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, ALU, and Controller.
- To implement each functional block using Verilog HDL with a modular and hierarchical design approach.
- To integrate all modules into a complete CPU datapath and control path.
- To verify the correctness of each module through individual testbenches.
- To evaluate the complete CPU by observing simulation waveforms and checking the execution of supported instructions.

By completing these objectives, the project provides practical experience in digital system design, hardware modeling, finite state machine design, and simulation-based verification.

1.3 Tools and Environment

The main tool used in this project is Vivado Design Suite. Vivado provides an integrated development environment for designing, simulating, and verifying digital hardware systems described using Verilog HDL. In this project, Vivado is used to create Verilog source files, develop testbenches, run simulations, and analyze signal waveforms. The source code is formatted, linted, and maintained using **Verible** and the SystemVerilog Language Server (SVLS) to ensure strict hardware coding conventions.

Each CPU component is implemented as an independent Verilog module and verified using a corresponding testbench. A custom **Python** script is implemented with Icarus Verilog (**iverilog**) serves as the primary compiler for rapid, iterative command-line simulations and unit testing. After unit-level verification, the modules are connected together in a top-level design to perform system-level simulation. The waveform viewer in Vivado is used to observe the behavior of internal signals, such as the program counter value, memory address, instruction register output, accumulator value, ALU result, and controller signals.

Vivado is suitable for this project because it supports hierarchical hardware design, Verilog-based modeling, testbench simulation, and detailed waveform analysis. These features allow the design to be verified systematically before further hardware synthesis or implementation.

1.4 Main Features of the CPU

The designed CPU supports a small but representative set of instructions. Since the opcode field is 3 bits wide, the CPU can define up to 8 instructions. These instructions include program control, arithmetic operations, logic operations, memory access, and conditional execution.

The main features of the CPU are summarized as follows:

- A 3-bit opcode field supporting eight basic instructions.
- A 5-bit operand field supporting 32 memory address locations.
- A Program Counter used to store and update the current instruction address.
- An Address Multiplexer used to select between the instruction address and operand address.
- A Memory module used to store both instructions and data.
- An Instruction Register used to hold and decode the current instruction.

- An Accumulator Register used to store intermediate and final computation results.
- An ALU used to perform arithmetic and logic operations.
- A Controller implemented as a finite state machine to generate control signals for instruction execution.
- A halt mechanism that stops program execution when the halt instruction is detected.

The CPU performs its operation through a sequence of basic stages: instruction fetch, instruction decode, operand access, execution, and result storage. This execution flow reflects the essential behavior of a real processor while remaining simple enough for analysis, implementation, and verification.

1.5 Report Organization

This report is organized into five main sections.

Section 1 introduces the project, including the project overview, objectives, tools and environment, main CPU features, and report structure.

Section 2 describes the theoretical background and the design of the CPU. This section presents the overall system architecture, block diagram, and detailed functions of each hardware module.

Section 3 discusses the implementation of the CPU using Verilog HDL. It explains the module structure, implementation approach, controller finite state machine, memory organization, and top-level integration.

Section 4 presents the testing and evaluation process. It includes unit-level testbenches, system-level simulation, waveform analysis, and functional evaluation of the CPU.

Section 5 concludes the report by summarizing the achieved results, discussing difficulties encountered during the project, and proposing possible future improvements.

2 System Design Approach

2.1 Theoretical Overview

Based on the project specification, the RISC CPU designed in this assignment features a 3-bit opcode and a 5-bit operand. This architecture supports 8 types of instructions (HLT, SKZ, ADD, AND, XOR, LDA, STO, JMP) and a 32-word memory address space. The processor operates synchronously with a clock signal and an active-high reset signal (the default reset state is `INST_ADDR`). Program execution stops upon encountering the HLT instruction.



By analyzing the controller state table and the functional requirements of each block, the CPU's operation flow can be divided into two primary phases spanning 8 clock cycles: the Fetch Phase and the Execution Phase.

Table 2.1: CPU Controller state and Control signals

Outputs	Phase								Notes
	INST_ADDR	INST_FETCH	INST_LOAD	IDLE	OP_ADDR	OP_FETCH	ALU_OP	STORE	
sel	1	1	1	1	0	0	0	0	ALU OP = 1 if opcode is ADD, AND, XOR or LDA
rd	0	1	1	1	0	ALUOP	ALUOP	ALUOP	
ld_ir	0	0	1	1	0	0	0	0	
halt	0	0	0	0	HALT	0	0	0	
inc_pc	0	0	0	0	1	0	SKZ && zero	0	
ld_ac	0	0	0	0	0	0	0	ALUOP	
ld_pc	0	0	0	0	0	0	JMP	JMP	
wr	0	0	0	0	0	0	0	STO	
data_e	0	0	0	0	0	0	STO	STO	

2.2 Top-level System Architecture

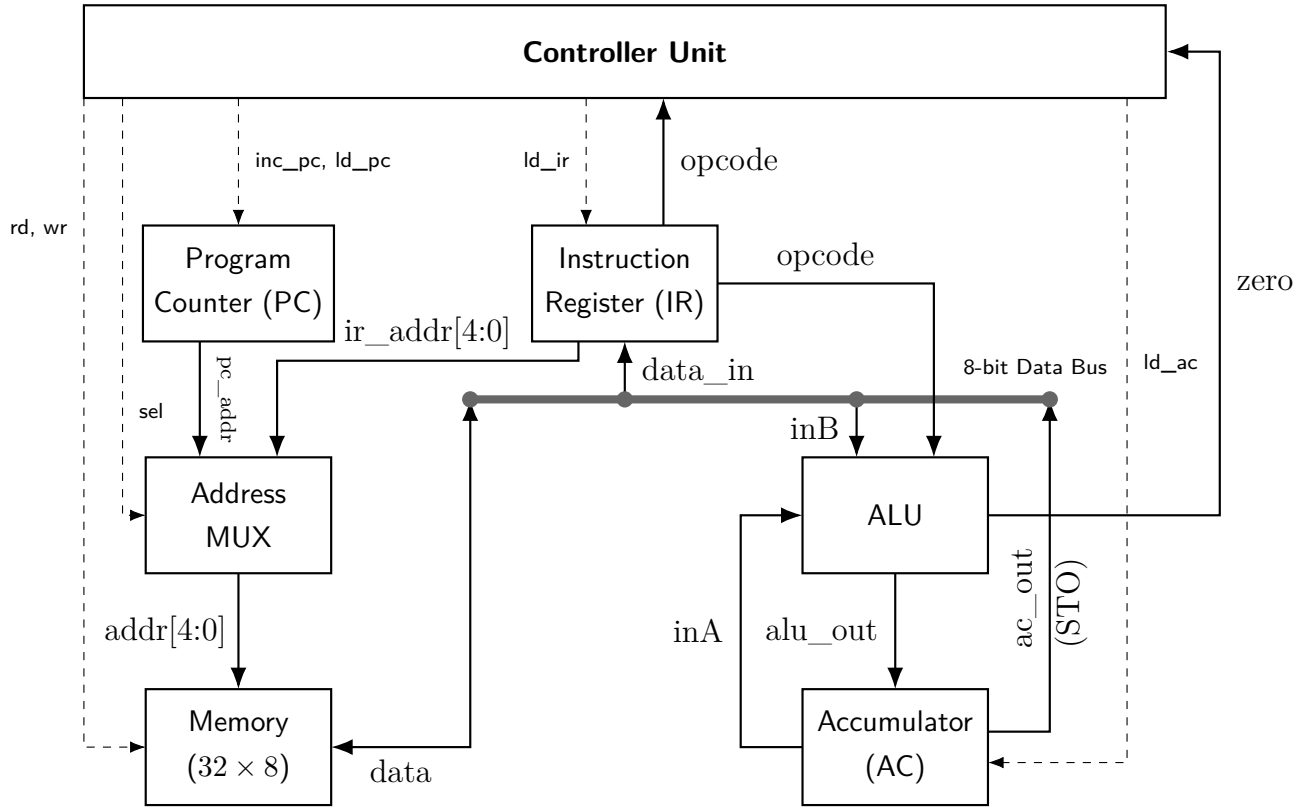


Figure 2.1: Top-Level Architecture of the RISC Processor

2.3 Fetch Phase

- **State 1: INST_ADDR.** The Program Counter (PC) provides the address of the current instruction. The Address Mux selects the PC address (`sel = 1`) to access the memory.
- **State 2: INST_FETCH.** The memory read signal (`rd = 1`) is asserted. The instruction data is retrieved from Memory.
- **State 3: INST_LOAD.** The fetched instruction is loaded into the Instruction Register (IR) via the `ld_ir = 1` control signal.
- **State 4: IDLE.** An intermediate wait state for the system to stabilize.

2.4 Execution Phase

- **State 5: OP_ADDR.** The IR splits the instruction into a 3-bit Opcode and a 5-bit Operand Address. The Address Mux switches to select the operand address from the IR (`sel`

= 0). Simultaneously, the PC increments (`inc_pc = 1`) to prepare for the next instruction cycle.

- **State 6: OP_FETCH.** If the instruction requires data from memory (e.g., ADD, AND, LDA), the CPU reads the data from the operand address in the Memory (`rd = 1`).
- **State 7: ALU_OP.** The Arithmetic Logic Unit (ALU) performs the required computation between the data fetched from Memory and the data currently held in the Accumulator (AC), based on the decoded Opcode. If the instruction is SKZ and the **zero** flag condition is met, the PC increments once more (`inc_pc = 1`) to skip the subsequent instruction.
- **State 8: STORE.** The result of the operation is either stored back into the Memory (for the ST0 instruction, asserting `wr = 1` and `data_e = 1`) or loaded into the Accumulator (for arithmetic and logic instructions, asserting `ld_ac = 1`).

2.5 Functional Blocks

Program Counter (PC)

- **Function:**
 - The counter is a crucial component used to track and count program instructions.
 - It can also be utilized to count program states.
 - The counter operates synchronously on the rising edge of the `clk` signal.
 - The `reset` signal is active-high, clearing the counter back to 0.
 - The counter has a width of 5 bits.
 - It features a load function to force an arbitrary address into the counter. If not loading, the counter increments normally.
- **Inputs:** `clk`, `rst`, `ld_pc`, `inc_pc`, `data_in`.
- **Outputs:** `pc_out`.

Address Multiplexer (Address MUX)

- **Function:**
 - The Address MUX selects between the instruction address (during the instruction fetch phase) and the operand address (during the instruction execution phase).
 - The MUX has a default data width of 5 bits.



- The width is parameterized to allow for easy modification if necessary.
- **Inputs:** `pc_addr`, `ir_addr`, `sel`.
- **Outputs:** `addr_out`.

Arithmetic Logic Unit (ALU)

- **Function:**
 - The ALU executes arithmetic and logical operations. The specific operation performed depends on the instruction's opcode.
 - The ALU executes 8 distinct operations on 8-bit operands (`inA` and `inB`).
 - It produces a 8-bit output and a 1-bit asynchronous **zero** flag.
 - The **zero** flag indicates whether the input `inA` is equal to 0.
- **Inputs:** `inA`, `inB`, `opcode`.
- **Outputs:** `alu_out`, `zero`.

Table 2.2: ALU Opcode Lookup Table

Opcode	Code	Operation Description	Output
HLT	000	Halts program execution.	inA
SKZ	001	Checks if the ALU result is equal to 0. If zero, it skips the next instruction; otherwise, execution continues normally.	inA
ADD	010	Adds the value in the Accumulator to the memory value at the instruction's address, returning the result to the Accumulator.	inA + inB
AND	011	Performs a bitwise AND on the Accumulator value and the memory value at the instruction's address.	inA AND inB
XOR	100	Performs a bitwise XOR on the Accumulator value and the memory value at the instruction's address.	inA XOR inB
LDA	101	Reads the value from the address in the instruction and loads it into the Accumulator.	inB
STO	110	Writes the Accumulator's data to the memory address specified in the instruction.	inA
JMP	111	Unconditional jump; jumps to the target address in the instruction and continues execution.	inA

Controller Unit

- **Function:** The central finite state machine that decodes instructions and dictates the operation of all other blocks via control signals.
- **Inputs:** clk, rst, opcode, zero.
- **Outputs:** sel, rd, ld_ir, halt, inc_pc, ld_ac, ld_pc, wr, data_e.

Table 2.3: Control Execution Scenarios by Opcode

Instruction Scenario	Active States	Expected Control Signals
HLT (000)	OP_ADDR	halt = 1
ADD, AND, XOR, LDA	OP_FETCH to STORE	rd = 1, ld_ac = 1 (at STORE phase)
STO (110)	ALU_OP, STORE	wr = 1, data_e = 1
JMP (111)	ALU_OP, STORE	ld_pc = 1

Instruction Register (IR)

- **Function:** As described in the reference documentation, it captures and holds the current instruction fetched from memory, splitting it into an opcode and an operand.
- **Inputs:** clk, rst, ld_ir, data_in.
- **Outputs:** opcode, operand.

Accumulator Register (AC)

- **Function:** As described in the reference documentation, it acts as the primary working register for ALU operations and data transfers.
- **Inputs:** clk, rst, ld_ac, data_in.
- **Outputs:** ac_out.

Memory

- **Function:**
 - The memory module stores both instructions and data.
 - It is organized as a 32×8 -bit memory, where the address width is 5 bits and each memory location stores 8 bits of data.
 - The memory uses a single bidirectional data bus, so read and write operations cannot occur at the same time.
 - The **rd** and **wr** control signals are used to enable memory read and memory write operations, respectively.
 - All memory operations are synchronized with the positive edge of the **clk** signal.



- **Inputs:** clk, rd, wr, addr.
- **Inout:** data.

3 Implementation

3.1 Implementation Strategy

The implementation follows a bottom-to-up design flow consistent with industrial RTL practice. The process went through 4 phases: specification review, RTL coding for each individual module, functional simulation, and unit testing for debugging to ensure any failures happened only in the current development module and that the hardware matched the intended behavior.

Throughout the implementation, the following coding style guidelines were enforced for readable, synthesizable, and maintainable Verilog source code: maintaining consistent indentation, naming conventions (`lower_case` variable casing), and comprehensive comments throughout the code.

3.2 Program Counter (PC)

Implementation Logic & Steps:

- Implemented a sequential logic block triggered by the positive edge of the clock.
- Implemented a synchronous active-high reset to clear the program counter.
- Implemented logic to load a new address into the program counter when `ld_pc` is active.
- Implemented logic to increment the program counter when `inc_pc` is active.

Source Code: PC.v:

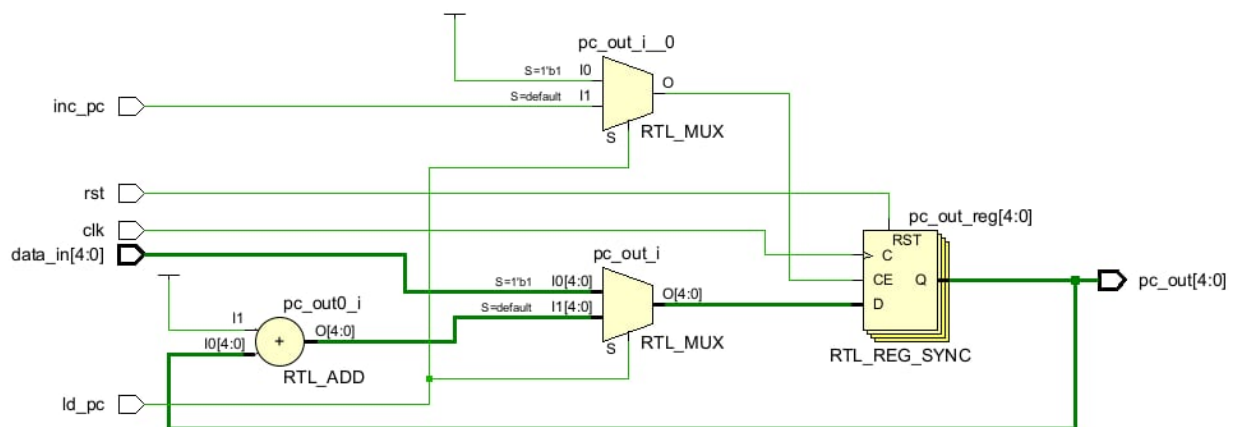
```
1 module PC (  
2     input      clk,  
3     input      rst,  
4     input      ld_pc,  
5     input      inc_pc,  
6     input      [4:0] data_in,  
7     output reg [4:0] pc_out  
8 );  
9  
10    always @(posedge clk) begin
```

```

11   if (rst) begin
12       pc_out <= 5'd0;
13   end else if (ld_pc) begin
14       pc_out <= data_in;
15   end else if (inc_pc) begin
16       pc_out <= pc_out + 5'd1;
17   end else begin
18       pc_out <= pc_out;
19   end
20 end
21
22 endmodule

```

RTL Schematic:



3.3 Address Multiplexer (Address MUX)

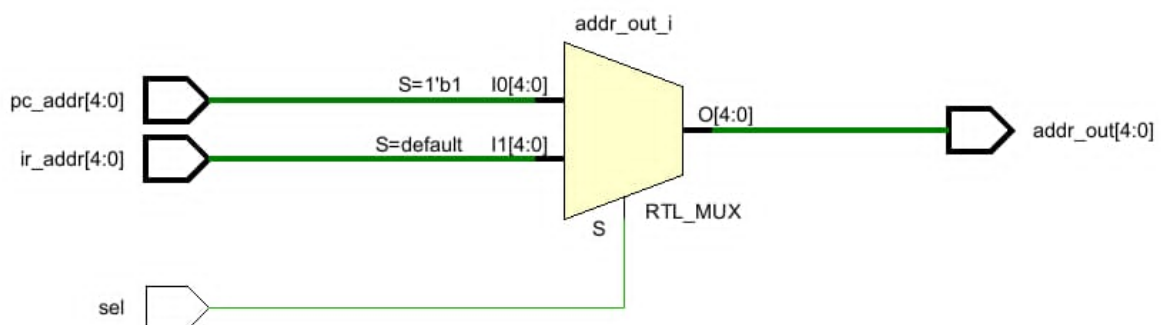
Implementation Logic & Steps:

- Implemented combinational logic to select between `pc_addr` and `ir_addr` based on the `sel` signal.
- Routed `pc_addr` to the output when `sel` is 1, and `ir_addr` when `sel` is 0.

Source Code: `ADD_MUX.v`:

```
1 module address_mux #(
2     parameter WIDTH = 5
3 ) (
4     input  [WIDTH-1:0] pc_addr,
5     input  [WIDTH-1:0] ir_addr,
6     input                    sel,
7     output [WIDTH-1:0] addr_out
8 );
9
10    assign addr_out = (sel) ? pc_addr : ir_addr;
11
12 endmodule
```

RTL Schematic:



3.4 Arithmetic Logic Unit (ALU)

Implementation Logic & Steps:

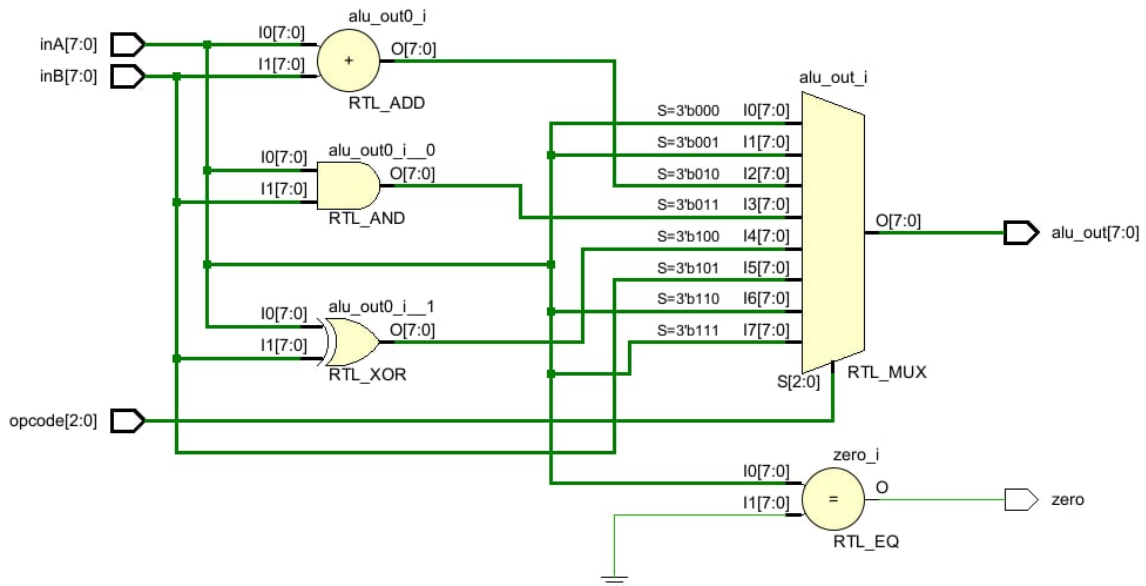
- Implemented combinational logic to set the zero flag if `inA` is 0.
- Implemented combinational logic to perform arithmetic and logic operations based on the opcode.
- Defined operations for HLT, SKZ, ADD, AND, XOR, LDA, STO, and JMP instructions.
- Assigned the combinational result to `alu_out`.

Source Code: ALU.v:



```
1 module ALU (  
2     input      [2:0] opcode,  
3     input      [7:0] inA,  
4     input      [7:0] inB,  
5     output reg [7:0] alu_out,  
6     output      zero  
7 );  
8     always @(*) begin  
9         case (opcode)  
10            3'b000: alu_out = inA;  
11            3'b001: alu_out = inA;  
12            3'b010: alu_out = inA + inB;  
13            3'b011: alu_out = inA & inB;  
14            3'b100: alu_out = inA ^ inB;  
15            3'b101: alu_out = inB;  
16            3'b110: alu_out = inA;  
17            3'b111: alu_out = inA;  
18            default: alu_out = 8'b00000000;  
19        endcase  
20    end  
21    assign zero = (inA == 8'b00000000);  
22  
23 endmodule
```

RTL Schematic:



3.5 Controller Unit

Implementation Logic & Steps:

- Defined state parameters for the 8-state FSM.
- Defined opcode parameters for the 8 instructions.
- Declared a state register.
- Implemented a sequential logic block for FSM state transitions (triggered by positive clock edges).
- Implemented a synchronous active-high reset to set the initial state.
- Implemented combinational logic to determine output control signals based on the current state and opcode.
- Ensured all output signals were assigned a default value to prevent unwanted latches.

Source Code: Controller.v:

```
1 module controller (
2     input      clk,
3     input      rst,
```



```
4      input      [2:0] opcode,
5      input      zero,
6      output reg  sel,
7      output reg  rd,
8      output reg  ld_ir,
9      output reg  halt,
10     output reg  inc_pc,
11     output reg  ld_ac,
12     output reg  ld_pc,
13     output reg  wr,
14     output reg  data_e
15
16 );
17     localparam INST_ADDR=3'd0,
18               INST_FETCH=3'd1,
19               INST_LOAD=3'd2,
20               IDLE=3'd3,
21               OP_ADDR=3'd4,
22               OP_FETCH=3'd5,
23               ALU_OP=3'd6,
24               STORE=3'd7;
25
26 // Opcodes
27     localparam HLT = 3'b000,
28               SKZ = 3'b001,
29               ADD = 3'b010,
30               AND = 3'b011,
31               XOR = 3'b100,
32               LDA = 3'b101,
33               STO = 3'b110,
34               JMP = 3'b111;
35
36     reg [2:0] state;
37     reg halted;
38
39     wire aluop;
```



```
40 assign aluop = (opcode == ADD) || (opcode == AND) || (opcode == XOR) ||  
    (opcode == LDA);  
41  
42 // State register  
43 always @(posedge clk) begin  
44     if (rst) begin  
45         state <= INST_ADDR;  
46         halted <= 1'b0;  
47     end else if (halted) begin  
48         state <= state;  
49         halted <= 1'b1;  
50     end else begin  
51         if ((state == OP_ADDR) && (opcode == HLT)) begin  
52             state <= OP_ADDR;  
53             halted <= 1'b1;  
54         end else begin  
55             state <= (state == STORE) ? INST_ADDR : state + 3'd1;  
56         end  
57     end  
58 end  
59  
60 // Output logic (combinational)  
61 always @(*) begin  
62     // Default values  
63     sel    = 1'b0;  
64     rd     = 1'b0;  
65     ld_ir  = 1'b0;  
66     halt   = 1'b0;  
67     inc_pc = 1'b0;  
68     ld_ac  = 1'b0;  
69     ld_pc  = 1'b0;  
70     wr     = 1'b0;  
71     data_e = 1'b0;  
72  
73     if (rst) begin  
74         sel = 1'b1;
```



```
75     end else if (halted) begin
76         halt = 1'b1;
77     end else begin
78         case (state)
79
80             INST_ADDR: begin
81                 sel = 1'b1;
82             end
83
84             INST_FETCH: begin
85                 sel = 1'b1;
86                 rd  = 1'b1;
87             end
88
89             INST_LOAD: begin
90                 sel  = 1'b1;
91                 rd   = 1'b1;
92                 ld_ir = 1'b1;
93             end
94
95             IDLE: begin
96                 sel  = 1'b1;
97                 rd   = 1'b1;
98                 ld_ir = 1'b1;
99             end
100
101             OP_ADDR: begin
102                 inc_pc = 1'b1;
103
104                 if (opcode == HLT) begin
105                     halt = 1'b1;
106                 end
107             end
108
109             OP_FETCH: begin
110                 rd = aluop;
```



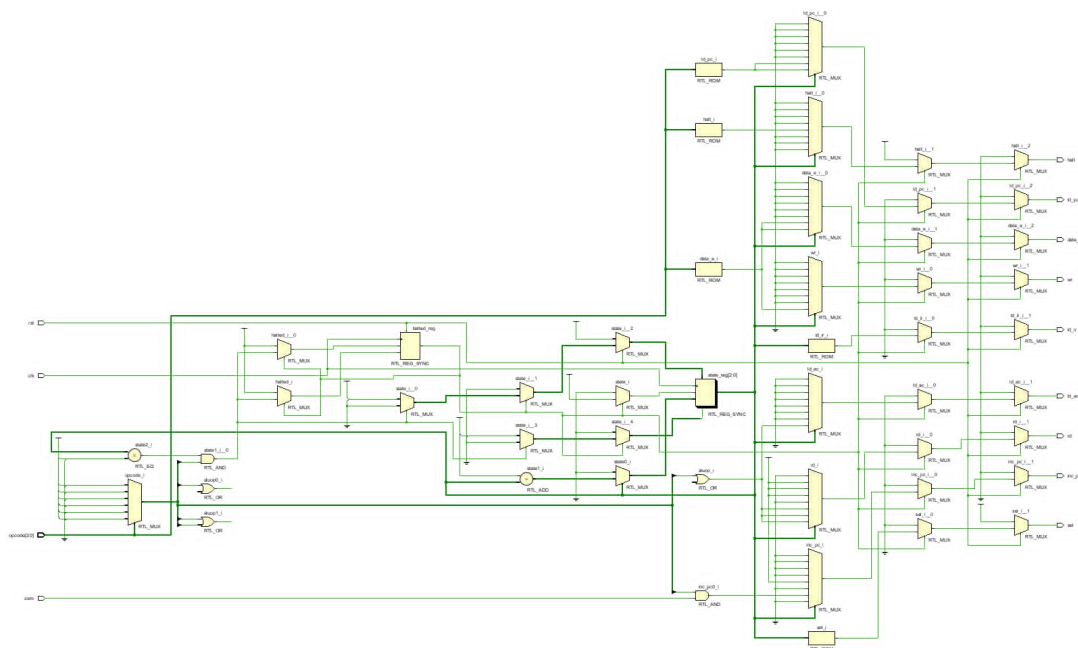
```
111     end
112
113     ALU_OP: begin
114         rd = aluop;
115
116         if (opcode == SKZ && zero) begin
117             inc_pc = 1'b1;
118         end
119
120         if (opcode == JMP) begin
121             ld_pc = 1'b1;
122         end
123
124         if (opcode == ST0) begin
125             data_e = 1'b1;
126         end
127     end
128
129     STORE: begin
130         rd    = aluop;
131         ld_ac = aluop;
132
133         if (opcode == JMP) begin
134             ld_pc = 1'b1;
135         end
136
137         if (opcode == ST0) begin
138             wr    = 1'b1;
139             data_e = 1'b1;
140         end
141     end
142
143     default: begin
144         sel    = 1'b0;
145         rd     = 1'b0;
146         ld_ir  = 1'b0;
```

```

147     halt    = 1'b0;
148     inc_pc  = 1'b0;
149     ld_ac   = 1'b0;
150     ld_pc   = 1'b0;
151     wr      = 1'b0;
152     data_e  = 1'b0;
153     end
154
155     endcase
156   end
157 end
158
159 endmodule

```

RTL Schematic:



3.6 Instruction Register (IR)

Implementation Logic & Steps:

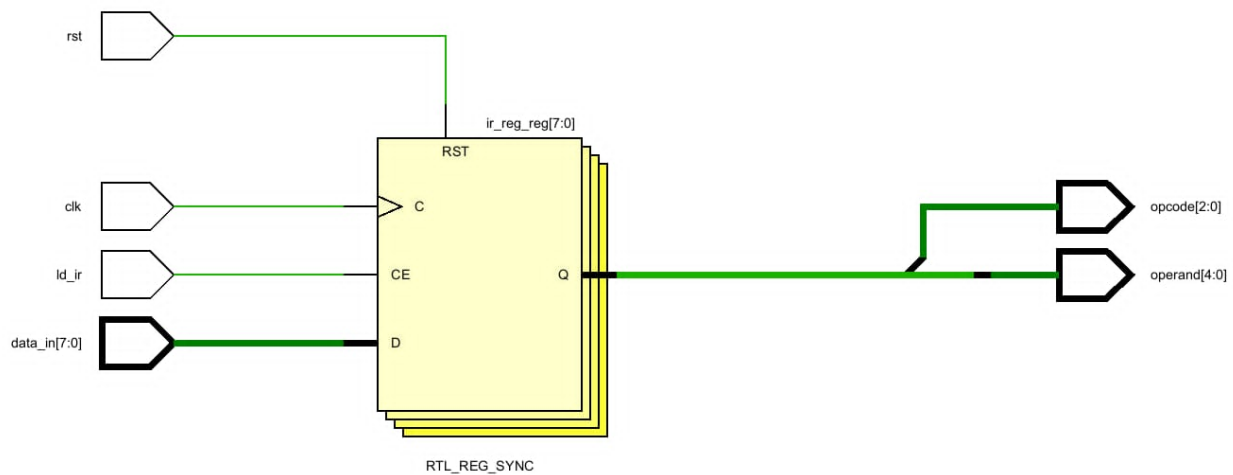
- Declared an internal register to store the instruction data.
- Implemented a sequential logic block triggered by the positive edge of the clock.

- Implemented a synchronous active-high reset to clear the register.
- Implemented logic to load `data_in` into the register when `ld_ir` is active.
- Implemented combinational logic to extract the 3-bit opcode from the stored instruction.
- Implemented combinational logic to extract the 5-bit operand from the stored instruction.

Source Code: IR.v:

```
1 module instruction_register (  
2     input      clk,  
3     input      rst,  
4     input      ld_ir,  
5     input  [7:0] data_in,  
6     output [2:0] opcode,  
7     output [4:0] operand  
8 );  
9  
10    reg [7:0] ir_reg;  
11  
12    always @(posedge clk) begin  
13        if (rst) begin  
14            ir_reg <= 8'b00000000;  
15        end else if (ld_ir) begin  
16            ir_reg <= data_in[7:0];  
17        end  
18    end  
19  
20    assign opcode  = ir_reg[7:5];  
21  
22    assign operand = ir_reg[4:0];  
23  
24 endmodule
```

RTL Schematic:



3.7 Accumulator Register (AC)

Implementation Logic & Steps:

- Implemented a sequential logic block triggered by the positive edge of the clock.
- Implemented a synchronous active-high reset to clear the accumulator.
- Implemented logic to load new data into the accumulator when the load signal is active.

Source Code: AC.v:

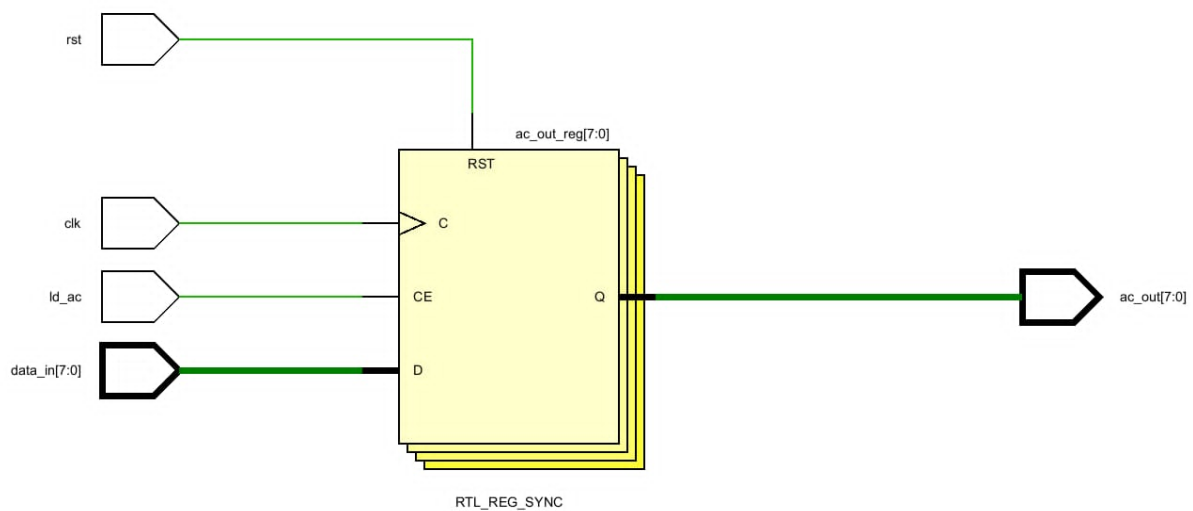
```

1 module accumulator (
2     input      clk,
3     input      rst,
4     input      [7:0] data_in,
5     input      ld_ac,
6     output reg [7:0] ac_out
7 );
8
9 always @(posedge clk) begin
10     if (rst) ac_out <= 8'b00000000;
11     else if (ld_ac) ac_out <= data_in;
12     else ac_out <= ac_out;
13 end
14

```


15 endmodule

RTL Schematic:



3.8 Memory

Implementation Logic & Steps:

- Declared a 2D array of registers to represent memory cells (32 cells of 8 bits).
- Declared a register to hold the output data.
- Implemented a sequential logic block triggered by the positive edge of the clock for writing data.
- Ensured writing only occurs when **wr** is active and **rd** is inactive.
- Implemented a sequential logic block triggered by the positive edge of the clock for reading data.
- Ensured reading only occurs when **rd** is active and **wr** is inactive.
- Implemented tri-state buffer logic for the bidirectional data bus.

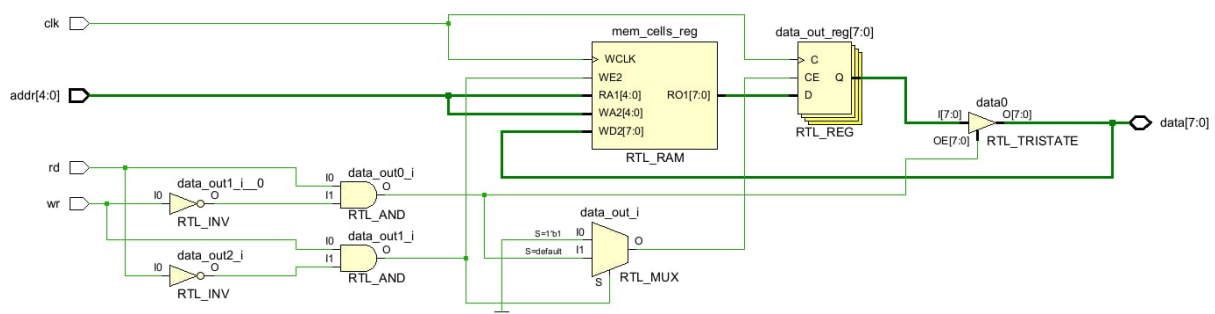
Source Code: Memory.v:

```

1 module Memory (
2     input      clk,
3     input      rd,
4     input      wr,
5     input [4:0] addr,
6     inout [7:0] data
7 );
8
9 reg [7:0] mem_cells[31:0];
10 reg [7:0] data_out;
11
12 always @(posedge clk) begin
13     if (wr && !rd) begin
14         mem_cells[addr] <= data;
15     end else if (rd && !wr) begin
16         data_out <= mem_cells[addr];
17     end
18 end
19
20 assign data = (rd && !wr) ? data_out : 8'hZZ;
21
22 endmodule

```

RTL Schematic:



3.9 Top-level Integration

Implementation Logic & Steps:

- Declared internal wires for connecting all the CPU submodules.
- Instantiated the Program Counter (PC) module and connected its ports.
- Instantiated the Address Multiplexer (MUX) module and connected its ports.
- Instantiated the ALU module and connected its ports.
- Instantiated the Controller module and connected its ports.
- Instantiated the Instruction Register (IR) module and connected its ports.
- Instantiated the Accumulator (AC) module and connected its ports.
- Instantiated the Memory module and connected its ports.
- Implemented tri-state buffer logic for the data bus to handle writing from the Accumulator to Memory.

Source Code: CPU.v:

```
1 module CPU (  
2     input  clk,  
3     input  rst,  
4     output halt  
5 );  
6  
7 // Control signals  
8 wire sel;  
9 wire rd;  
10 wire ld_ir;  
11 wire inc_pc;  
12 wire ld_ac;  
13 wire ld_pc;  
14 wire wr;  
15 wire data_e;  
16  
17 // Datapath signals  
18 wire [4:0] pc_out;  
19 wire [4:0] operand;  
20 wire [4:0] mem_addr;  
21
```



```
22  wire [2:0] opcode;
23
24  wire [7:0] data_bus;
25  wire [7:0] ac_out;
26  wire [7:0] alu_out;
27
28  wire zero;
29
30  assign data_bus = (data_e && !rd) ? ac_out : 8'hZZ;
31
32  PC u_pc (
33      .clk      (clk),
34      .rst      (rst),
35      .ld_pc    (ld_pc),
36      .inc_pc   (inc_pc),
37      .data_in  (operand),
38      .pc_out   (pc_out)
39  );
40
41  address_mux #(
42      .WIDTH(5)
43  ) u_address_mux (
44      .pc_addr  (pc_out),
45      .ir_addr  (operand),
46      .sel      (sel),
47      .addr_out (mem_addr)
48  );
49
50  Memory u_memory (
51      .clk (clk),
52      .rd  (rd),
53      .wr  (wr),
54      .addr(mem_addr),
55      .data(data_bus)
56  );
57
```



```
58  instruction_register u_ir (  
59      .clk      (clk),  
60      .rst      (rst),  
61      .ld_ir   (ld_ir),  
62      .data_in (data_bus),  
63      .opcode  (opcode),  
64      .operand (operand)  
65  );  
66  
67  accumulator u_ac (  
68      .clk      (clk),  
69      .rst      (rst),  
70      .data_in (alu_out),  
71      .ld_ac   (ld_ac),  
72      .ac_out  (ac_out)  
73  );  
74  
75  ALU u_alu (  
76      .opcode (opcode),  
77      .inA    (ac_out),  
78      .inB    (data_bus),  
79      .alu_out (alu_out),  
80      .zero    (zero)  
81  );  
82  
83  controller u_controller (  
84      .clk      (clk),  
85      .rst      (rst),  
86      .opcode  (opcode),  
87      .zero    (zero),  
88      .sel      (sel),  
89      .rd       (rd),  
90      .ld_ir   (ld_ir),  
91      .halt     (halt),  
92      .inc_pc  (inc_pc),  
93      .ld_ac   (ld_ac),
```

```
94     .ld_pc (ld_pc),  
95     .wr     (wr),  
96     .data_e(data_e)  
97 );  
98  
99 endmodule
```

RTL Schematic:

4 Testing Simulation and Validation

4.1 Program Counter (PC)

Testbench code: test_001/PC_tb.v:

```
1  `timescale 1ns / 1ps  
2  
3  // =====  
4  // PC_tb.v - Testbench for Program Counter (PC)  
5  // PC is 5-bit according to the assignment: 32 memory addresses.  
6  // Covers:  
7  //   TC1: Synchronous reset  
8  //   TC2: Increment  
9  //   TC3: Load  
10 //   TC4: Priority rst > ld_pc > inc_pc  
11 //   TC5: Hold  
12 //   TC6: 5-bit wrap-around  
13 //   TC7: Jump & increment  
14 //   TC8: Reset from non-zero value  
15 //   TC9: Data noise immunity  
16 // =====  
17 module PC_tb;  
18  
19     reg        clk;  
20     reg        rst;  
21     reg        ld_pc;  
22     reg        inc_pc;
```

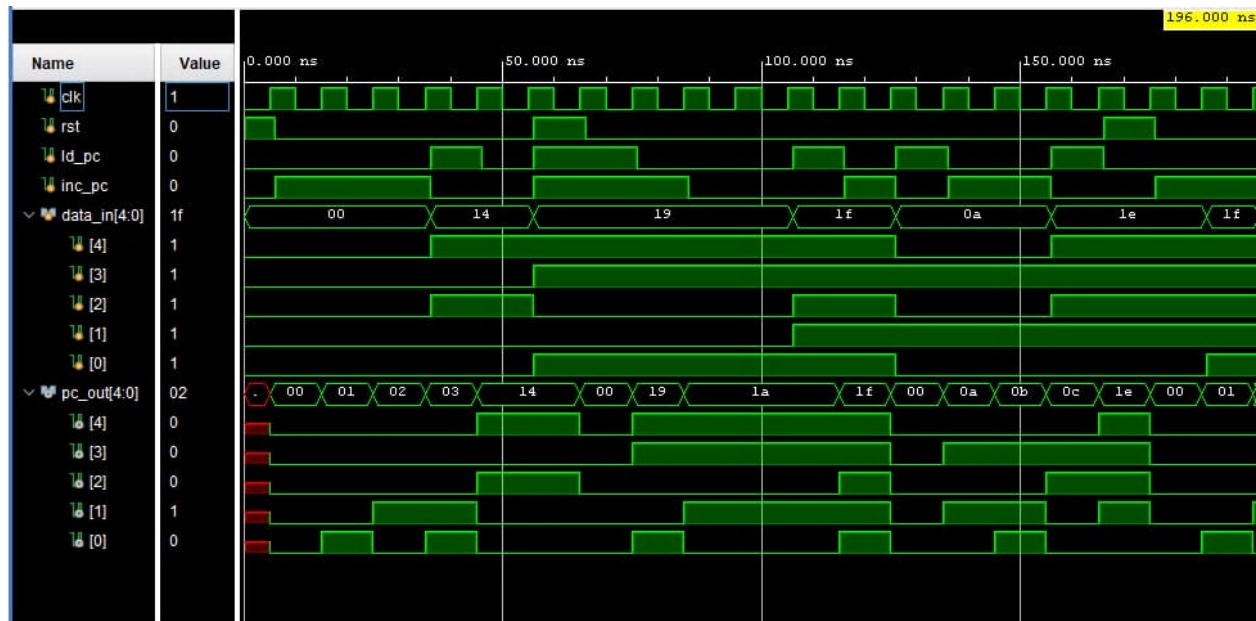
```
23 reg [4:0] data_in;
24 wire [4:0] pc_out;
25
26 PC uut (
27     .clk      (clk),
28     .rst      (rst),
29     .ld_pc    (ld_pc),
30     .inc_pc   (inc_pc),
31     .data_in  (data_in),
32     .pc_out   (pc_out)
33 );
34
35 initial clk = 0;
36 always #5 clk = ~clk;
37
38 task tick;
39     input [63:0] cycle;
40     begin
41         @(posedge clk);
42         #1;
43         $display("cycle=%0d | rst=%b | ld_pc=%b | inc_pc=%b | data_in=%0d |
44                 pc_out=%0d", cycle, rst,
45                     ld_pc, inc_pc, data_in, pc_out);
46     end
47 endtask
48
49 initial begin
50     rst = 1;
51     ld_pc = 0;
52     inc_pc = 0;
53     data_in = 5'd0;
54
55     $display("--- TC1: Reset ---");
56     tick(1); // pc_out = 0
57     rst = 0;
```

```
58     $display("--- TC2: Increment ---");
59     inc_pc = 1;
60     tick(2); // pc_out = 1
61     tick(3); // pc_out = 2
62     tick(4); // pc_out = 3
63     inc_pc = 0;
64
65     $display("--- TC3: Load ---");
66     data_in = 5'd20;
67     ld_pc = 1;
68     tick(5); // pc_out = 20
69     ld_pc = 0;
70     tick(6); // hold 20
71
72     $display("--- TC4: Priority rst > ld_pc > inc_pc ---");
73     data_in = 5'd25;
74     rst = 1;
75     ld_pc = 1;
76     inc_pc = 1;
77     tick(7); // reset dominates -> 0
78     rst = 0;
79     tick(8); // ld_pc dominates -> 25
80     ld_pc = 0;
81     tick(9); // inc_pc -> 26
82     inc_pc = 0;
83
84     $display("--- TC5: Hold State ---");
85     tick(10); // hold 26
86     tick(11); // hold 26
87
88     $display("--- TC6: 5-bit Wrap-around ---");
89     data_in = 5'd31;
90     ld_pc = 1;
91     tick(12); // pc_out = 31
92     ld_pc = 0;
93     inc_pc = 1;
```



```
94     tick(13); // pc_out wraps to 0
95     inc_pc = 0;
96
97     $display("--- TC7: Jump & Increment ---");
98     data_in = 5'd10;
99     ld_pc   = 1;
100    tick(14); // pc_out = 10
101    ld_pc   = 0;
102    inc_pc = 1;
103    tick(15); // 11
104    tick(16); // 12
105    inc_pc = 0;
106
107    $display("--- TC8: Reset from Non-Zero Value ---");
108    data_in = 5'd30;
109    ld_pc   = 1;
110    tick(17); // pc_out = 30
111    ld_pc   = 0;
112    rst     = 1;
113    tick(18); // pc_out = 0
114    rst     = 0;
115
116    $display("--- TC9: Data Noise Immunity ---");
117    inc_pc = 1;
118    tick(19); // pc_out = 1
119    data_in = 5'd31;
120    tick(20); // pc_out = 2, not affected by data_in
121    inc_pc = 0;
122
123    $display("--- DONE ---");
124    $finish;
125 end
126
127 endmodule
```

Waveform:



4.2 Address Multiplexer (Address MUX)

Testbench code: test_002/AM_tb.v:

```

1  `timescale 1ns / 1ps
2
3  // =====
4  // AM_tb.v - Testbench for Address Mux
5  // Default WIDTH is 5 because CPU memory has 32 addresses.
6  // Covers:
7  //   TC1: Select pc_addr
8  //   TC2: Select ir_addr
9  //   TC3: Toggle sel
10 //   TC4: Parameter override
11 //   TC5: Stability
12 //   TC6: Edge case addresses
13 //   TC7: Async response
14 // =====
15 module AM_tb;
16
17     reg [4:0] pc_addr;
18     reg [4:0] ir_addr;

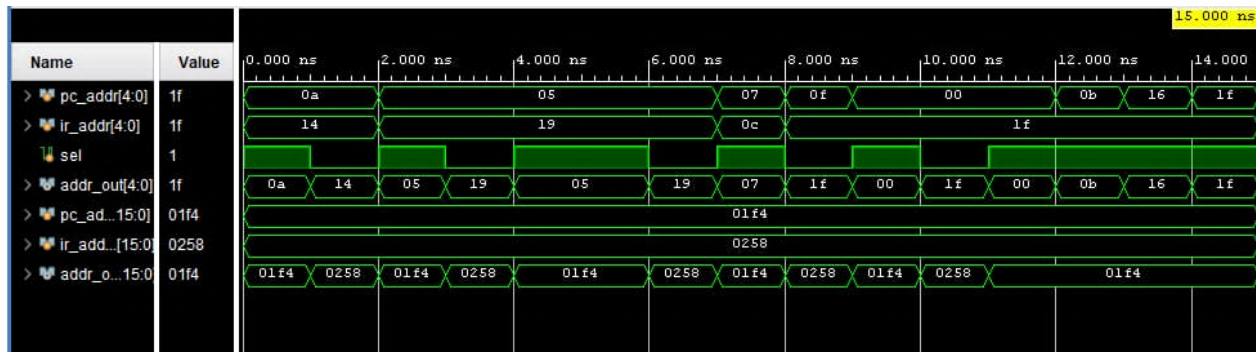
```

```
19 reg      sel;
20 wire [4:0] addr_out;
21
22 address_mux uut (
23     .pc_addr (pc_addr),
24     .ir_addr (ir_addr),
25     .sel      (sel),
26     .addr_out(addr_out)
27 );
28
29 reg [15:0] pc_addr_16;
30 reg [15:0] ir_addr_16;
31 wire [15:0] addr_out_16;
32
33 address_mux #(
34     .WIDTH(16)
35 ) uut_16 (
36     .pc_addr (pc_addr_16),
37     .ir_addr (ir_addr_16),
38     .sel      (sel),
39     .addr_out(addr_out_16)
40 );
41
42 task display_state;
43     input [8*5:1] tc_name;
44     begin
45         #1;
46         $display("%s | sel=%b | pc_addr=%0d | ir_addr=%0d | addr_out=%0d",
47             tc_name, sel, pc_addr,
48             ir_addr, addr_out);
49     end
50 endtask
51
52 initial begin
53     pc_addr = 5'd10;
54     ir_addr = 5'd20;
```

```
54     sel = 0;
55
56     pc_addr_16 = 16'd500;
57     ir_addr_16 = 16'd600;
58
59     $display("--- TC1: Select pc_addr ---");
60     sel = 1;
61     display_state("TC1  ");
62
63     $display("--- TC2: Select ir_addr ---");
64     sel = 0;
65     display_state("TC2  ");
66
67     $display("--- TC3: Toggle sel ---");
68     pc_addr = 5'd5;
69     ir_addr = 5'd25;
70     sel = 1;
71     display_state("TC3.1");
72     sel = 0;
73     display_state("TC3.2");
74     sel = 1;
75     display_state("TC3.3");
76
77     $display("--- TC4: Parameter Override (WIDTH=16) ---");
78     sel = 1;
79     #1;
80     $display("TC4.1 | sel=%b | pc_addr_16=%0d | ir_addr_16=%0d |
81             addr_out_16=%0d", sel, pc_addr_16,
82             ir_addr_16, addr_out_16);
83     sel = 0;
84     #1;
85     $display("TC4.2 | sel=%b | pc_addr_16=%0d | ir_addr_16=%0d |
86             addr_out_16=%0d", sel, pc_addr_16,
87             ir_addr_16, addr_out_16);
88
89     $display("--- TC5: Stability Check ---");
```

```
88     sel = 1;
89     pc_addr = 5'd7;
90     ir_addr = 5'd12;
91     display_state("TC5.1");
92     sel = 0;
93     pc_addr = 5'd15;
94     ir_addr = 5'd31;
95     display_state("TC5.2");
96
97     $display("--- TC6: Edge Case Addresses ---");
98     sel = 1;
99     pc_addr = 5'd0;
100    ir_addr = 5'd31;
101    display_state("TC6.1");
102    sel = 0;
103    display_state("TC6.2");
104
105    $display("--- TC7: Async Response ---");
106    sel = 1;
107    #1;
108    pc_addr = 5'd11;
109    display_state("TC7.1");
110    pc_addr = 5'd22;
111    display_state("TC7.2");
112    pc_addr = 5'd31;
113    display_state("TC7.3");
114
115    $display("--- DONE ---");
116    $finish;
117 end
118
119 endmodule
```

Waveform:



4.3 Arithmetic Logic Unit (ALU)

Testbench code: test_003/ALU_tb.v:

```

1  `timescale 1ns / 1ps
2
3  // =====
4  // ALU_tb.v - Testbench for Arithmetic Logic Unit (ALU)
5  // Covers 7 scenarios:
6  //   TC1: Zero Status Flag
7  //   TC2: Arithmetic & Logic
8  //   TC3: Transfer Operations
9  //   TC4: Combinational Property
10 //   TC5: Boundary Cases
11 //   TC6: Complex Bitwise Logic
12 //   TC7: Zero Flag Independence
13 // =====
14 module ALU_tb;
15
16     reg [7:0] inA;
17     reg [7:0] inB;
18     reg [2:0] opcode;
19
20     wire [7:0] alu_out;
21     wire      zero;
22
23     // DUT

```



```
24  ALU uut (
25      .inA    (inA),
26      .inB    (inB),
27      .opcode (opcode),
28      .alu_out(alu_out),
29      .zero    (zero)
30  );
31
32  task display_state;
33      input [8*6:1] tc_name;
34      begin
35          #1;
36          $display("%s | opcode=%b | inA=%0d | inB=%0d | zero=%b | alu_out=%0d",
37                  tc_name, opcode, inA,
38                      inB, zero, alu_out);
39      end
40  endtask
41
42  initial begin
43      // Initialize
44      inA = 0;
45      inB = 0;
46      opcode = 3'b000;
47      #10;
48
49      // TC1: Zero Status Flag
50      $display("\n--- TC1: Zero Status Flag ---");
51
52      inA = 8'd0;
53      display_state("TC1.1"); // zero = 1
54
55      inA = 8'd50;
56      display_state("TC1.2"); // zero = 0
57
58      // TC2: Arithmetic & Logic Operations
```



```
59     $display("\n--- TC2: Arithmetic & Logic ---");
60
61     inA = 8'd100;
62     inB = 8'd25;
63
64     // ADD
65     opcode = 3'b010;
66     display_state("ADD"); // 125
67
68     // AND
69     opcode = 3'b011;
70     display_state("AND"); // 0
71
72     // XOR
73     opcode = 3'b100;
74     display_state("XOR"); // 125
75
76     // LDA
77     opcode = 3'b101;
78     display_state("LDA"); // 25
79
80
81     // TC3: Transfer Instructions
82     $display("\n--- TC3: HLT / SKZ / STO / JMP ---");
83
84     inA = 8'd200;
85     inB = 8'd100;
86
87     // HLT
88     opcode = 3'b000;
89     display_state("HLT"); // 200
90
91     // SKZ
92     opcode = 3'b001;
93     display_state("SKZ"); // 200
94
```




```
95 // ST0
96 opcode = 3'b110;
97 display_state("ST0"); // 200
98
99 // JMP
100 opcode = 3'b111;
101 display_state("JMP"); // 200
102
103
104 // TC4: Combinational Property
105 $display("\n--- TC4: Combinational Property ---");
106
107 opcode = 3'b010; // ADD
108
109 inA = 8'd10;
110 inB = 8'd20;
111 #1;
112
113 $display("TC4.1 | alu_out=%0d (Expected 30)", alu_out);
114
115 // Change inputs immediately
116 inA = 8'd50;
117 inB = 8'd10;
118 #1;
119
120 $display("TC4.2 | alu_out=%0d (Expected 60)", alu_out);
121
122 // TC5: Boundary Cases
123 $display("\n--- TC5: Boundary Cases ---");
124
125 opcode = 3'b010; // ADD
126
127 inA = 8'hFF;
128 inB = 8'h01;
129 display_state("OVF1"); // Expect 0
130
```

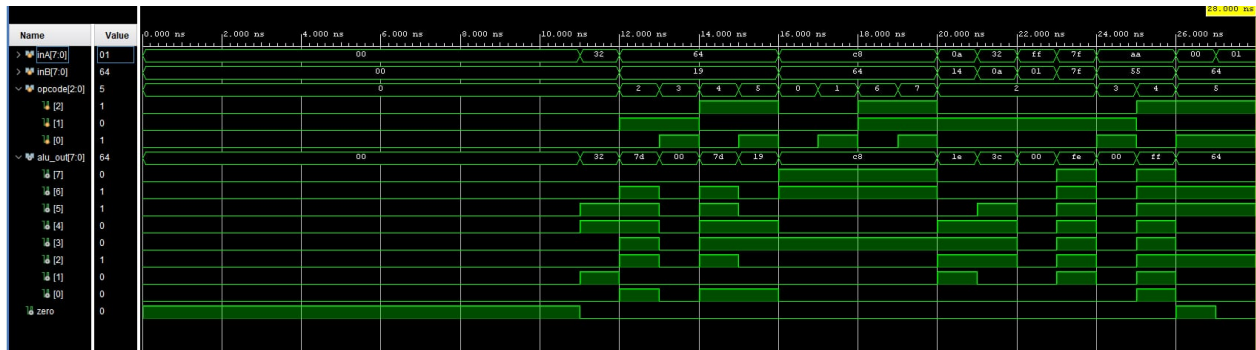


```
131     inA = 8'h7F;
132     inB = 8'h7F;
133     display_state("OVF2"); // Expect 254
134
135     // TC6: Complex Bitwise Logic
136     $display("\n--- TC6: Complex Bitwise Logic ---");
137
138     inA = 8'hAA; // 10101010
139     inB = 8'h55; // 01010101
140
141     opcode = 3'b011;
142     display_state("AND_C"); // 0
143
144     opcode = 3'b100;
145     display_state("XOR_C"); // 255
146
147
148     // TC7: Zero Flag Independence
149     $display("\n--- TC7: Zero Flag Independence ---");
150
151     opcode = 3'b101; // LDA
152
153     inA = 8'd0;
154     inB = 8'd100;
155     #1;
156     $display("TC7.1 | zero=%b | alu_out=%0d", zero, alu_out);
157
158     inA = 8'd1;
159     inB = 8'd100;
160     #1;
161     $display("TC7.2 | zero=%b | alu_out=%0d", zero, alu_out);
162
163     $display("\n--- ALL TESTS FINISHED ---");
164     $finish;
165
166 end
```

167

168 endmodule

Waveform:



4.4 Controller Unit

Testbench code: test_004/Controller_tb.v:

```

1 `timescale 1ns / 1ps
2
3 // =====
4 // Controller_tb.v - Testbench for Controller
5 // Covers:
6 //   TC1: Reset
7 //   TC2: Fetch phase
8 //   TC3: ADD execution
9 //   TC4: ST0 execution
10 //   TC5: JMP execution
11 //   TC6: HLT stable halt
12 //   TC7: SKZ with zero=0 and zero=1
13 // =====
14 module Controller_tb;
15
16     reg        clk;
17     reg        rst;
18     reg [2:0] opcode;
19     reg        zero;
20

```

```
21 wire sel, rd, ld_ir, halt, inc_pc, ld_ac, ld_pc, wr, data_e;
22
23 controller uut (
24     .clk    (clk),
25     .rst    (rst),
26     .opcode(opcode),
27     .zero   (zero),
28     .sel    (sel),
29     .rd     (rd),
30     .ld_ir  (ld_ir),
31     .halt   (halt),
32     .inc_pc (inc_pc),
33     .ld_ac  (ld_ac),
34     .ld_pc  (ld_pc),
35     .wr     (wr),
36     .data_e(data_e)
37 );
38
39 initial clk = 0;
40 always #5 clk = ~clk;
41
42 task tick;
43     begin
44         @(posedge clk);
45         #1;
46     end
47 endtask
48
49 task reset_controller;
50     begin
51         rst = 1;
52         repeat (2) @(posedge clk);
53         #1;
54         rst = 0;
55     end
56 endtask
```

```
57
58 task display_state;
59     input [8*14:1] state_name;
60     begin
61         $display(
62             "%s | state=%3b opcode=%3b zero=%b | sel=%b rd=%b ld_ir=%b halt=%b
63             inc_pc=%b ld_ac=%b ld_pc=%b wr=%b data_e=%b",
64             state_name, uut.state, opcode, zero, sel, rd, ld_ir, halt, inc_pc,
65             ld_ac, ld_pc, wr,
66             data_e);
67     end
68 endtask
69
70 initial begin
71     opcode = 3'b000;
72     zero   = 1'b0;
73     rst    = 1'b1;
74
75     reset_controller();
76
77     $display("--- TC1: System Reset / INST_ADDR ---");
78     display_state("0:INST_ADDR  ");
79
80     $display("--- TC2: Fetch Phase ---");
81     opcode = 3'b010; // ADD
82     display_state("0:INST_ADDR  ");
83     tick();
84     display_state("1:INST_FETCH ");
85     tick();
86     display_state("2:INST_LOAD  ");
87     tick();
88     display_state("3:IDLE      ");
89
90     $display("--- TC3: Execution Phase (ADD) ---");
91     tick();
92     display_state("4:OP_ADDR    ");
```

```
91     tick();
92     display_state("5:OP_FETCH    ");
93     tick();
94     display_state("6:ALU_OP      ");
95     tick();
96     display_state("7:STORE       ");
97     tick();
98     display_state("0:INST_ADDR   ");
99
100     $display("--- TC4: Execution Phase (STO) ---");
101     reset_controller();
102     opcode = 3'b110; // STO
103     tick();
104     tick();
105     tick();
106     tick();
107     display_state("4:OP_ADDR     ");
108     tick();
109     display_state("5:OP_FETCH    ");
110     tick();
111     display_state("6:ALU_OP      ");
112     tick();
113     display_state("7:STORE       ");
114
115     $display("--- TC5: Execution Phase (JMP) ---");
116     reset_controller();
117     opcode = 3'b111; // JMP
118     tick();
119     tick();
120     tick();
121     tick();
122     display_state("4:OP_ADDR     ");
123     tick();
124     display_state("5:OP_FETCH    ");
125     tick();
126     display_state("6:ALU_OP      ");
```



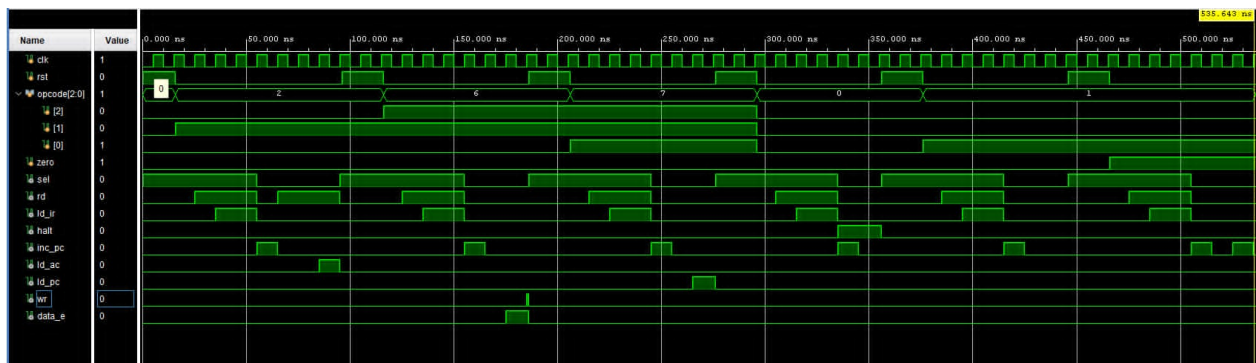
```
127     tick();
128     display_state("7:STORE      ");
129
130     $display("--- TC6: Execution Phase (HLT, stable halt) ---");
131     reset_controller();
132     opcode = 3'b000; // HLT
133     tick();
134     tick();
135     tick();
136     tick();
137     display_state("4:OP_ADDR/HLT ");
138     tick();
139     display_state("HOLD_HALT_1   ");
140     tick();
141     display_state("HOLD_HALT_2   ");
142
143     $display("--- TC7.1: SKZ zero=0, no skip ---");
144     reset_controller();
145     opcode = 3'b001; // SKZ
146     zero    = 1'b0;
147     tick();
148     tick();
149     tick();
150     tick();
151     display_state("4:OP_ADDR      ");
152     tick();
153     display_state("5:OP_FETCH      ");
154     tick();
155     display_state("6:ALU_OP        "); // inc_pc should be 0
156     tick();
157     display_state("7:STORE        ");
158
159     $display("--- TC7.2: SKZ zero=1, skip next instruction ---");
160     reset_controller();
161     opcode = 3'b001; // SKZ
162     zero    = 1'b1;
```

```

163     tick();
164     tick();
165     tick();
166     tick();
167     display_state("4:OP_ADDR      ");
168     tick();
169     display_state("5:OP_FETCH     ");
170     tick();
171     display_state("6:ALU_OP       "); // inc_pc should be 1
172     tick();
173     display_state("7:STORE        ");
174
175     $display("--- DONE ---");
176     $finish;
177 end
178
179 endmodule

```

Waveform:



4.5 Instruction Register (IR)

Testbench code: test_005/IR_tb.v:

```

1  `timescale 1ns / 1ps
2
3  // =====
4  // IR_tb.v - Testbench for Instruction Register (IR)

```




```
5 // Covers 7 scenarios described in the project spec:
6 // TC1: System Reset
7 // TC2: Load Instruction
8 // TC3: Hold Value
9 // TC4: Extract Opcode and Operand
10 // TC5: Reset Priority
11 // TC6: Bus Noise Immunity
12 // TC7: Instruction Decoding
13 // =====
14
15 module IR_tb;
16 // - Signals -----
17 reg clk;
18 reg rst;
19 reg ld_ir;
20 reg [7:0] data_in;
21
22 wire [2:0] opcode;
23 wire [4:0] operand;
24
25 // - DUT instantiation -----
26 instruction_register uut (
27     .clk    (clk),
28     .rst    (rst),
29     .ld_ir  (ld_ir),
30     .data_in(data_in),
31     .opcode (opcode),
32     .operand(operand)
33 );
34
35 // - Clock: 10 ns period -----
36 initial begin
37     clk = 0;
38     forever #5 clk = ~clk;
39 end
40
```

```
41 // - Task: tick one clock and display state -----
42 task display_state;
43     input [8*5:1] tc_name;
44     begin
45         @(posedge clk);
46         #1; // Wait after posedge for data to update
47         $display("%s | rst=%b | ld_ir=%b | data_in=0x%02X | opcode=%b |
48             operand=%b", tc_name, rst,
49                 ld_ir, data_in, opcode, operand);
50     end
51 endtask
52
53 initial begin
54     // Init
55     rst = 1;
56     ld_ir = 0;
57     data_in = 8'd0;
58     #1;
59     // - TC1: System Reset -----
60     $display("--- TC1: System Reset ---");
61     display_state("TC1.1"); // Expect opcode=0, operand=0
62
63     rst = 0; // De-assert reset
64
65     // - TC2: Load Instruction -----
66     $display("--- TC2: Load Instruction ---");
67     ld_ir = 1;
68     // Instruction 1: Opcode = 101 (LDA), Operand = 10101
69     // Combined: [7:5] = 101, [4:0] = 10101 -> 1011_0101 = 0xB5
70     data_in = 8'hB5;
71     display_state("TC2.1"); // Expect: opcode=101, operand=10101
72
73     // Instruction 2: Opcode = 010 (ADD), Operand = 00011
74     // Combined: [7:5] = 010, [4:0] = 00011 -> 0100_0011 = 0x43
75     data_in = 8'h43;
76     display_state("TC2.2"); // Expect: opcode=010, operand=00011
```

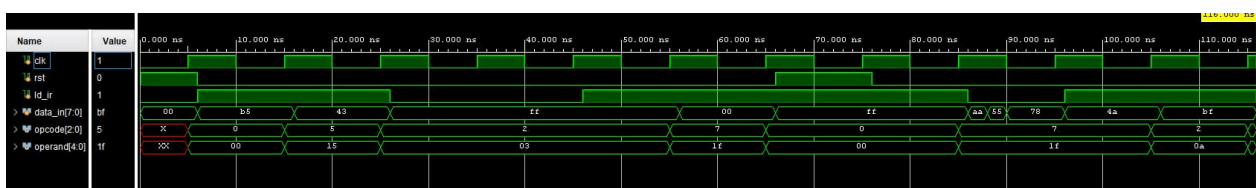


```
76
77 // - TC3: Hold Value -----
78 $display("--- TC3: Hold Value ---");
79 ld_ir    = 0; // De-assert load instruction signal
80 data_in  = 8'hFF; // Change data on bus
81 display_state("TC3.1"); // Expect: opcode=010, operand=00011 (holds
    value)
82 display_state("TC3.2"); // Expect: opcode=010, operand=00011
83
84 // - TC4: Extract Opcode and Operand -----
85 $display("--- TC4: Extract Opcode and Operand ---");
86 // Instruction 3: Opcode = 111 (JMP), Operand = 11111
87 // Combined: [7:5] = 111, [4:0] = 11111 -> 1111_1111 = 0xFF
88 ld_ir    = 1;
89 data_in  = 8'hFF;
90 display_state("TC4.1"); // Expect: opcode=111, operand=11111
91
92 // Instruction 4: Opcode = 000 (HLT), Operand = 00000
93 data_in  = 8'h00;
94 display_state("TC4.2"); // Expect: opcode=000, operand=00000
95
96 // - TC5: Reset Priority -----
97 $display("--- TC5: Reset Priority ---");
98 ld_ir    = 1;
99 data_in  = 8'hFF; // Attempt to load all 1s
100 rst      = 1; // But assert reset simultaneously
101 display_state("TC5.1"); // Expect: opcode=000, operand=00000
102 rst      = 0;
103 display_state("TC5.2"); // After reset is de-asserted, FF data is
    loaded (opcode=111, op=11111)
104
105 // - TC6: Bus Noise Immunity -----
106 // Change data_in continuously when ld_ir = 0
107 $display("--- TC6: Bus Noise Immunity ---");
108 ld_ir    = 0;
109 data_in  = 8'hAA;
```



```
110     #2;
111     data_in = 8'h55;
112     #2;
113     data_in = 8'h78;
114     display_state("TC6.1"); // Expect: holds value from TC5.2 (111, 1111)
115
116     // - TC7: Instruction Decoding -----
117     $display("--- TC7: Instruction Decoding ---");
118     ld_ir    = 1;
119     // Load ADD (010) Operand (01010)
120     data_in = 8'h4A; // 4A = 010_01010
121     display_state("TC7.1"); // Expect: opcode=010, operand=01010
122
123     // Load LDA (101) Operand (11111)
124     data_in = 8'hBF; // BF = 101_11111
125     display_state("TC7.2"); // Expect: opcode=101, operand=11111
126
127     $display("--- DONE ---");
128     $finish;
129 end
130 endmodule
```

Waveform:



4.6 Accumulator Register (AC)

Testbench code: test_006/AC_tb.v:

```
1 `timescale 1ns / 1ps
2
3 // =====
4 // AC_tb.v - Testbench for Accumulator (AC)
```

```
5 // Covers 8 scenarios:
6 //   TC1: System Reset
7 //   TC2: Load Data
8 //   TC3: Data Stability
9 //   TC4: Feedback to ALU
10 //   TC5: Interaction with Data Bus
11 //   TC6: Reset Priority
12 //   TC7: Boundary Values
13 //   TC8: Rapid Toggling
14 // =====
15 module AC_tb;
16
17 // - Signals -----
18 reg      clk;
19 reg      rst;
20 reg      ld_ac;
21 reg [7:0] data_in;
22 wire [7:0] ac_out;
23
24 // - DUT instantiation -----
25 accumulator uut (
26     .clk      (clk),
27     .rst      (rst),
28     .ld_ac    (ld_ac),
29     .data_in  (data_in),
30     .ac_out   (ac_out)
31 );
32
33 // - Clock: 10 ns period -----
34 initial begin
35     clk = 0;
36     forever #5 clk = ~clk;
37 end
38
39 // - Task: tick one clock and display state -----
40 task display_state;
```

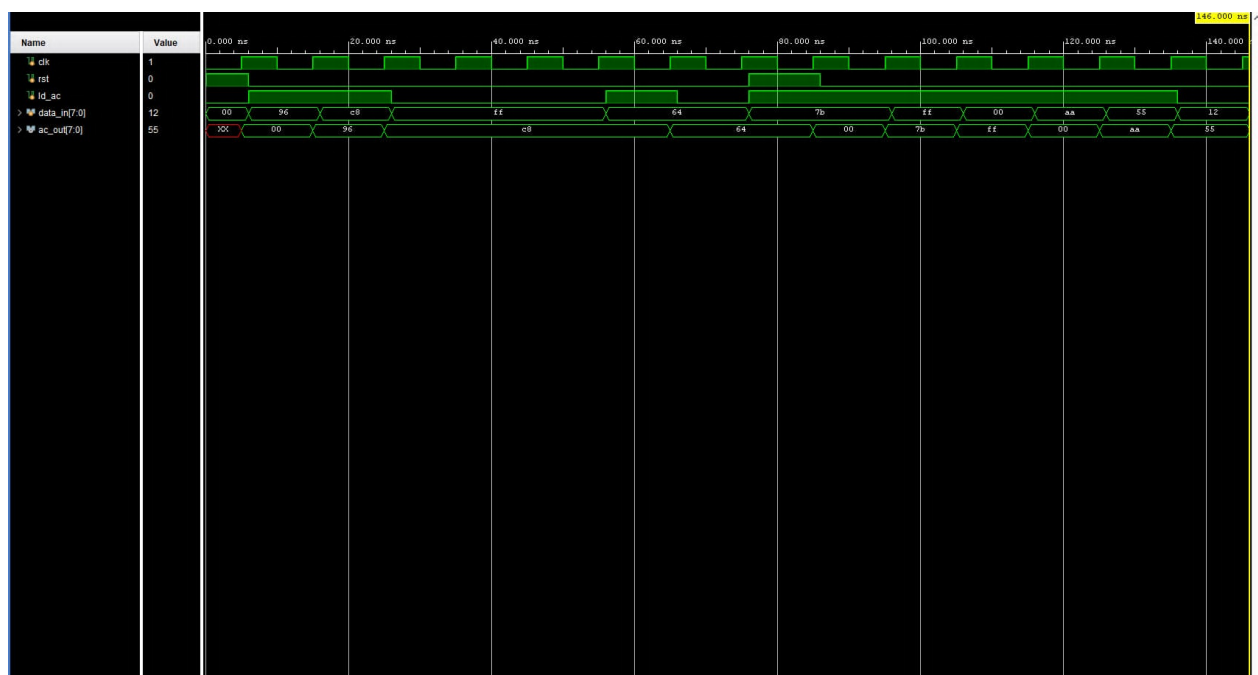
```
41     input [8*5:1] tc_name;
42     begin
43         @(posedge clk);
44         #1; // Wait after posedge for data to stabilize
45         $display("%s | rst=%b | ld_ac=%b | data_in=%0d | ac_out=%0d", tc_name,
46             rst, ld_ac, data_in,
47                 ac_out);
48     end
49 endtask
50
51 initial begin
52     // Init
53     rst = 1;
54     ld_ac = 0;
55     data_in = 8'd0;
56     #1;
57
58     // - TC1: System Reset -----
59     $display("--- TC1: System Reset ---");
60     display_state("TC1 "); // Expect: ac_out = 0
61     rst = 0;
62
63     // - TC2: Load Data (ALU Result) -----
64     $display("--- TC2: Load Data (ALU Result) ---");
65     ld_ac = 1;
66     data_in = 8'd150; // Assume result from ALU is 150
67     display_state("TC2.1"); // Expect: ac_out = 150
68
69     data_in = 8'd200;
70     display_state("TC2.2"); // Expect: ac_out = 200
71
72     // - TC3: Data Stability (Idle/Fetch) -----
73     $display("--- TC3: Data Stability (Idle/Fetch) ---");
74     ld_ac = 0;
75     data_in = 8'd255; // Change input data
76     display_state("TC3.1"); // Expect: ac_out holds 200
```

```
76 display_state("TC3.2"); // Expect: ac_out holds 200
77
78 // - TC4: Feedback to ALU -----
79 $display("--- TC4: Feedback to ALU ---");
80 // This case verifies that ac_out (inA of ALU) is always ready
81 display_state("TC4 "); // Expect: ac_out = 200
82
83 // - TC5: Interaction with Data Bus -----
84 $display("--- TC5: Interaction with Data Bus ---");
85 // Simulate loading a new value in preparation to write to Memory
86 ld_ac = 1;
87 data_in = 8'd100;
88 display_state("TC5.1"); // Expect: ac_out = 100
89 ld_ac = 0;
90 display_state("TC5.2"); // Value holds stable during write cycle
91
92 // - TC6: Reset Priority -----
93 $display("--- TC6: Reset Priority Check ---");
94 ld_ac = 1;
95 data_in = 8'd123;
96 rst = 1; // Assert Reset while load is active
97 display_state("TC6.1"); // Expect: ac_out = 0 (Reset wins)
98 rst = 0;
99 display_state("TC6.2"); // Expect: ac_out = 123 (Normal load resumes)
100
101 // - TC7: Boundary Values -----
102 $display("--- TC7: Boundary Values ---");
103 ld_ac = 1;
104 data_in = 8'hFF; // All 1s
105 display_state("TC7.1");
106 data_in = 8'h00; // All 0s
107 display_state("TC7.2");
108
109 // - TC8: Rapid Toggling -----
110 $display("--- TC8: Rapid Toggling ---");
111 ld_ac = 1;
```



```
112 data_in = 8'hAA;
113 display_state("TC8.1");
114 data_in = 8'h55;
115 display_state("TC8.2");
116 ld_ac = 0;
117 data_in = 8'h12;
118 display_state("TC8.3"); // Expect: Holds 8'h55
119
120 $display("---- DONE ----");
121 $finish;
122 end
123 endmodule
```

Waveform:



4.7 Memory

Testbench code: test_007/Memory_tb.v:

```
1 `timescale 1ns / 1ps
2
3 // =====
```

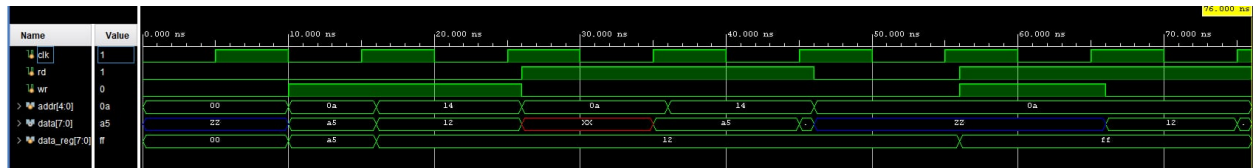



```
4 // Memory_tb.v - Testbench for Memory
5 // Covers 5 scenarios described in the project spec:
6 //   TC1: Write Data
7 //   TC2: Read Data
8 //   TC3: High Impedance Check
9 //   TC4: Conflict Prevention
10 //   TC5: Data Persistence
11 // =====
12 module Memory_tb;
13
14 // - Signals -----
15 reg      clk;
16 reg      rd;
17 reg      wr;
18 reg [4:0] addr; // 5-bit address for 32 Memory cells
19 wire [7:0] data; // Bidirectional bus
20
21 // Bus control signals from Testbench
22 reg [7:0] data_reg;
23
24 // Inout control logic: Write only when wr=1 and rd=0
25 assign data = (wr && !rd) ? data_reg : 8'hZZ;
26
27 // - DUT instantiation -----
28 Memory uut (
29     .clk (clk),
30     .rd  (rd),
31     .wr  (wr),
32     .addr(addr),
33     .data(data)
34 );
35
36 // - Clock: 10 ns period -----
37 initial begin
38     clk = 0;
39     forever #5 clk = ~clk;
```

```
40     end
41
42     // - Task: tick one clock and display state -----
43     task display_state;
44         input [8*5:1] tc_name;
45         begin
46             @(posedge clk);
47             #1; // Wait after posedge for data to stabilize
48             $display("%s | rd=%b | wr=%b | addr=%0d | data=0x%02X", tc_name, rd,
49                 wr, addr, data);
50         end
51     endtask
52
53     initial begin
54         // Init
55         rd      = 0;
56         wr      = 0;
57         addr    = 0;
58         data_reg = 0;
59         #10;
60
61         // - TC1: Write Data -----
62         $display("--- TC1: Write Data ---");
63         wr      = 1;
64         rd      = 0;
65         addr    = 5'd10;
66         data_reg = 8'hA5;
67         display_state("TC1.1"); // Write to Memory cell 10
68
69         addr    = 5'd20;
70         data_reg = 8'h12;
71         display_state("TC1.2"); // Write to Memory cell 20
72         wr = 0;
73
74         // - TC2: Read Data -----
75         $display("--- TC2: Read Data ---");
```

```
75     wr    = 0;
76     rd    = 1;
77     addr = 5'd10;
78     display_state("TC2.1"); // Expect: A5
79     addr = 5'd20;
80     display_state("TC2.2"); // Expect: 12
81
82     // - TC3: High Impedance Check -----
83     $display("--- TC3: High Impedance Check ---");
84     wr    = 0;
85     rd    = 0; // No read, no write
86     addr = 5'd10;
87     display_state("TC3  "); // Expect: data = ZZ
88
89     // - TC4: Conflict Prevention -----
90     $display("--- TC4: Conflict Prevention ---");
91     // Per spec: Simultaneous read and write not allowed
92     wr      = 1;
93     rd      = 1;
94     addr    = 5'd10;
95     data_reg = 8'hFF;
96     display_state("TC4  "); // Expect: System must handle safely (usually
97                               high-Z or priority)
98
99     // - TC5: Data Persistence -----
100    $display("--- TC5: Data Persistence ---");
101    wr    = 0;
102    rd    = 1;
103    addr = 5'd10;
104    display_state("TC5  "); // Confirm data in cell 10 is still A5 after
105                               conflict
106
107    $display("--- DONE ---");
108    $finish;
109 end
110 endmodule
```

Waveform:



4.8 Top-level Integration

Testbench code: test_008/CPU_tb.v:

```

1  `timescale 1ns / 1ps
2
3  // =====
4  // CPU_tb.v - Top-Level Testbench for Integration (CPU)
5  //   - LDA 20
6  //   - ADD 21
7  //   - STO 22
8  //   - HLT
9  // =====
10 module CPU_tb;
11
12   // - Signals -----
13   reg clk;
14   reg rst;
15   wire halt;
16
17   // - DUT instantiation -----
18   CPU uut (
19       .clk (clk),
20       .rst (rst),
21       .halt(halt)
22   );
23
24   // - Clock: 10 ns period -----
25   initial clk = 0;
26   always #5 clk = ~clk;
27
28   // - Task: display CPU state -----

```

```
29 task display_cpu_state;
30     begin
31         $display("time=%t | PC=%2d | State=%3b | Op=%3b | AC=%0d | Halt=%b",
32             $time, uut.u_pc.pc_out,
33             uut.u_controller.state, uut.opcode, uut.u_ac.ac_out, halt);
34     end
35 endtask
36
37 // - Load Program & Control Simulation -----
38 initial begin
39     // 1. Load specific test program
40     // LDA 20, ADD 21, STO 22, HLT
41     uut.u_memory.mem_cells[0] = 8'hB4; // LDA 20 (Opcode 101, Operand
42     // 10100)
43     uut.u_memory.mem_cells[1] = 8'h55; // ADD 21 (Opcode 010, Operand
44     // 10101)
45     uut.u_memory.mem_cells[2] = 8'hD6; // STO 22 (Opcode 110, Operand
46     // 10110)
47     uut.u_memory.mem_cells[3] = 8'h00; // HLT (Opcode 000, Operand
48     // 00000)
49
50     // 2. Load input data
51     uut.u_memory.mem_cells[20] = 8'd5;
52     uut.u_memory.mem_cells[21] = 8'd3;
53     uut.u_memory.mem_cells[22] = 8'd0; // Location to store result
54
55     // 3. Reset system
56     rst = 1;
57     repeat (2) @(posedge clk);
58     #1;
59     rst = 0;
60
61     $display("--- CPU INTEGRATION TEST STARTED ---");
62 end
63
64 // Monitor state at each positive clock edge
```



```
60  always @(posedge clk) begin
61      if (!rst) display_cpu_state();
62  end
63
64  // Final result check after HLT
65  initial begin
66      wait (halt === 1'b1);
67      repeat (2) @(posedge clk);
68
69      $display("--- FINAL RESULT CHECK ---");
70      $display("Mem[22] (Expected 8): %0d", uut.u_memory.mem_cells[22]);
71
72      if (uut.u_memory.mem_cells[22] == 8) $display("--- TEST STATUS: PASS
73      ---");
74      else $display("--- TEST STATUS: FAIL ---");
75
76      $finish;
77  end
78
79  // Safety Timeout
80  initial #10000 $finish;
81 endmodule
```

test_009/CPU2_tb.v:

```
1  `timescale 1ns / 1ps
2
3  // =====
4  // CPU2_tb.v - Top-Level Testbench for Integration (CPU)
5  // Covers execution of a comprehensive test program evaluating:
6  //   - LDA, ADD, STO sequence
7  //   - Conditional SKZ branching
8  //   - Unconditional JMP branching
9  //   - Bitwise XOR and AND operations
10 //   - HLT assertion
11 // =====
```



```
12 module CPU2_tb;
13
14 // - Signals -----
15 reg clk;
16 reg rst;
17 wire halt;
18
19 // - DUT instantiation -----
20 CPU uut (
21     .clk (clk),
22     .rst (rst),
23     .halt(halt)
24 );
25
26 // - Clock: 10 ns period -----
27 initial clk = 0;
28 always #5 clk = ~clk;
29
30 // - Task: display CPU state -----
31 // Prints out whenever PC changes or an instruction cycle finishes (8
32 // phases)
33 task display_cpu_state;
34     begin
35         $display("time=%t | PC=%2d | State=%3b | Op=%3b | AC=%0d | Halt=%b",
36             $time, uut.u_pc.pc_out,
37             uut.u_controller.state, uut.opcode, uut.u_ac.ac_out, halt);
38     end
39 endtask
40
41 // - Load Program & Control Simulation -----
42 initial begin
43     // 1. Load comprehensive test program into Memory
44     // Program executes: (5 + 3), XOR 5, AND 3, tests SKZ/JMP
45     uut.u_memory.mem_cells[0] = 8'hB4; // LDA 20 -> AC = 5
46     uut.u_memory.mem_cells[1] = 8'h55; // ADD 21 -> AC = 8
47     uut.u_memory.mem_cells[2] = 8'hD6; // STO 22 -> Mem[22] = 8
```



```
46    uut.u_memory.mem_cells[3] = 8'h20;    // SKZ    -> AC=8 (!=0) -> No jump
47    uut.u_memory.mem_cells[4] = 8'hE6;    // JMP 6   -> PC jumps to 6
48    uut.u_memory.mem_cells[5] = 8'h55;    // ADD 21 -> (Skipped due to JMP)
49    uut.u_memory.mem_cells[6] = 8'h94;    // XOR 20 -> 8 XOR 5 = 13
50    uut.u_memory.mem_cells[7] = 8'h75;    // AND 21 -> 13 AND 3 = 1
51    uut.u_memory.mem_cells[8] = 8'hD7;    // STO 23 -> Mem[23] = 1
52    uut.u_memory.mem_cells[9] = 8'hB8;    // LDA 24 -> AC = 0 (Data at 24 is
    0)
53    uut.u_memory.mem_cells[10] = 8'h20;   // SKZ     -> AC=0 -> Skip next
    instruction
54    uut.u_memory.mem_cells[11] = 8'hD9;   // STO 25 -> (Skipped due to SKZ)
55    uut.u_memory.mem_cells[12] = 8'h00;   // HLT     -> Halt
56
57    // 2. Load operand data
58    uut.u_memory.mem_cells[20] = 8'd5;
59    uut.u_memory.mem_cells[21] = 8'd3;
60    uut.u_memory.mem_cells[22] = 8'd0;
61    uut.u_memory.mem_cells[23] = 8'd0;
62    uut.u_memory.mem_cells[24] = 8'd0;
63
64    // 3. Reset system
65    rst = 1;
66    repeat (2) @(posedge clk);
67    #1;
68    rst = 0;
69
70    $display("--- CPU INTEGRATION TEST STARTED ---");
71    end
72
73    // Monitor state at each positive clock edge
74    always @(posedge clk) begin
75        if (!rst) display_cpu_state();
76    end
77
78    // Final result check after HLT
79    initial begin
```



```
80     wait (halt === 1'b1);
81     repeat (2) @(posedge clk);
82
83     $display("--- FINAL RESULT CHECK ---");
84     $display("Mem[22] (Expected 8): %0d", uut.u_memory.mem_cells[22]);
85     $display("Mem[23] (Expected 1): %0d", uut.u_memory.mem_cells[23]);
86
87     if (uut.u_memory.mem_cells[22] == 8 && uut.u_memory.mem_cells[23] == 1)
88         $display("--- TEST STATUS: PASS ---");
89     else $display("--- TEST STATUS: FAIL ---");
90
91     $finish;
92 end
93
94 // Safety Timeout
95 initial #10000 $finish;
96
97 endmodule
```

Waveform:

4.9 Resource, Timing, and Power Evaluation

In addition to functional verification, the CPU was evaluated at the RTL level in terms of resource usage, timing behavior, and expected power characteristics. The design follows a simple accumulator-based RISC architecture, so the datapath remains compact and includes only the required modules: the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, ALU, and Controller.

Resource usage is mainly determined by the storage elements and combinational logic in the RTL design. The processor uses a 5-bit Program Counter, an 8-bit Instruction Register, an 8-bit Accumulator Register, a 3-bit controller state register, and a 32×8 -bit memory. The main combinational blocks are the Address MUX, the ALU, and the controller output logic.

Timing behavior is defined by the controller FSM. The controller executes instructions through eight states: INST_ADDR, INST_FETCH, INST_LOAD, IDLE, OP_ADDR, OP_FETCH, ALU_OP, and STORE. Therefore, a normal instruction requires eight clock cycles to complete. With the 10 ns clock period used in simulation, the execution time of a normal instruction is 80 ns. Control-flow instructions such as SKZ and JMP may update the Program Counter, while HLT stops the processor when the halt condition is reached.



Table 4.1: Resource, Timing, and Power Evaluation

Metric	Evaluation	Description
Instruction width	8 bits	Includes a 3-bit opcode and a 5-bit operand.
Memory organization	32×8 bits	Provides 32 memory locations, each storing 8 bits of data.
Datapath width	8 bits	Used by the ALU, Accumulator, and shared data bus.
Address width	5 bits	Supports addressing of 32 memory locations.
Controller states	8 states	Forms one complete instruction cycle under normal execution.
Simulation clock period	10 ns	Clock period used in the Verilog testbenches.
Normal instruction time	80 ns	Calculated from eight clock cycles with a 10 ns clock period.
Sequential resources	PC, IR, AC, FSM, Memory	Store the program address, instruction, accumulator value, control state, and memory contents.
Combinational resources	ALU, Address MUX, control logic	Perform arithmetic and logic operations, address selection, and control signal generation.
Power evaluation	RTL-level qualitative assessment	Expected to be low due to the small datapath, limited memory size, and simple control logic.

From the evaluation, the CPU satisfies the expected timing behavior of the specified multi-cycle controller. The design completes each normal instruction in eight clock cycles and correctly updates the Program Counter for control-flow instructions. Because the datapath is only 8 bits wide and the memory contains only 32 locations, the hardware complexity remains low. Exact post-synthesis resource utilization and power consumption depend on the target device and synthesis settings; therefore, this report provides an RTL-level evaluation based on the implemented architecture and simulation results.

5 Conclusion and Future Work

5.1 Conclusion

In this assignment, our group designed and implemented a simple RISC processor using Verilog HDL. The processor follows an accumulator-based architecture and uses an 8-bit instruction format, consisting of a 3-bit opcode and a 5-bit operand field. Based on this format, the CPU supports eight instructions: HLT, SKZ, ADD, AND, XOR, LDA, STO, and JMP. The design also includes a 5-bit program counter and a 32-location memory space, which are consistent with the project specification.

The processor was developed using a modular design approach. Core components, including the Program Counter, Address Multiplexer, Memory, Instruction Register, Accumulator Register, ALU, and Controller, were implemented as independent Verilog modules and then integrated into the top-level CPU module. This organization made the datapath and control path clearer, while also making simulation, debugging, and system integration more manageable.

Key part of the design was the Controller, which was implemented as a finite state machine with eight states: `INST_ADDR`, `INST_FETCH`, `INST_LOAD`, `IDLE`, `OP_ADDR`, `OP_FETCH`, `ALU_OP`, and `STORE`. These states coordinate the main phases of instruction execution, including instruction fetch, decoding, operand access, ALU operation, accumulator update, memory write-back, program counter update, and halt control. Through simulation, the processor was verified to execute the supported instructions correctly at both module and system levels.

The main strength of the design is its clear separation between datapath components and control logic. Since each module was tested independently before being connected in the complete CPU, functional errors were easier to identify and correct. In addition, the use of both module-level and CPU-level testbenches helped confirm the correctness of individual blocks as well as the overall instruction execution flow.

However, the current processor still has several limitations. The instruction set is relatively small and does not support more complex arithmetic or comparison operations. The 5-bit address bus also limits the memory space to only 32 locations, which restricts the size of executable programs. Moreover, the controller follows a fixed multi-state execution sequence, meaning that some simple instructions may require more clock cycles than necessary. The project also focuses mainly on functional simulation, while synthesis results, timing delay, resource utilization, and power consumption have not yet been analyzed in detail.

Overall, the project successfully achieved the main requirements of designing a simple RISC processor using Verilog HDL. The processor supports the required 3-bit opcode and 5-bit operand instruction format, implements all required functional blocks, and executes the eight specified instructions. Each module was verified using an individual testbench, and the complete CPU was validated through top-level simulation. Although the design is simple, it demonstrates the essential operation of a processor, including instruction fetching, decoding, operand access, ALU execution, memory access, and program control.

5.2 Difficulties Encountered

Several difficulties were encountered during the implementation and testing of the processor. The first challenge was controlling the timing of internal signals across different execution states. Since each instruction is completed through multiple clock cycles, control signals for memory access, instruction loading, accumulator loading, program counter updating, and memory writing

had to be activated in the correct order. Any incorrect control signal could cause the CPU to fetch the wrong instruction, update the wrong register, or access memory at the wrong time.

Another challenge was working with the shared bidirectional data bus. The same bus is used for both reading data from memory and writing data back to memory, so the design had to ensure that only one component drives the bus at a time. This was especially important for the store instruction, where the accumulator value must be placed on the data bus only during the correct write phase.

The program counter behavior also required careful verification. For normal instructions, the program counter increments sequentially. However, branch-related instructions change this behavior: the skip instruction may require an additional increment when the zero condition is satisfied, while the jump instruction loads a new address directly into the program counter. These cases were tested carefully because small timing errors could change the execution flow of the whole program.

The final difficulty was integrating all modules into one complete CPU. Although each block passed its individual testbench, the full processor introduced additional timing interactions between the controller, memory, instruction register, accumulator, ALU, and program counter. These issues were resolved by checking simulation waveforms and comparing the internal signal transitions with the expected instruction execution sequence.

5.3 Future Plan

Based on these limitations, several improvements can be considered for future development. First, the instruction set can be extended to improve the flexibility of the processor. Additional instructions such as SUB, MUL, DIV, shift operations, and comparison instructions would allow the CPU to support more diverse computational tasks. For example, comparison and shift instructions would make the processor more capable of handling conditional operations and bit-level data manipulation.

Second, the memory system can be expanded by increasing the address width. Since the current 5-bit address bus only supports 32 memory locations, a wider address bus would allow the processor to access a larger program and data memory space. This improvement would make it possible to execute longer instruction sequences and store more intermediate data during program execution.

Third, the Controller can be optimized to reduce unnecessary execution cycles. In the current design, instructions follow a fixed sequence of states, even when some operations do not require all stages. Future versions could use instruction-dependent state transitions so that simpler instructions, such as HLT or JMP, complete in fewer cycles. More advanced control techniques, including pipelining and hazard handling, could also be explored to improve instruction throughput.

Fourth, verification should be extended with more comprehensive test programs. Although module-level and CPU-level simulations were performed, additional cases should be tested, such as repeated jump operations, consecutive store instructions, SKZ behavior when the accumulator is zero and non-zero, and boundary memory access at the lowest and highest addresses. These cases would help evaluate the robustness of the processor under less straightforward execution scenarios.

Finally, synthesis and implementation analysis should be performed to evaluate the hardware cost of the design. Resource utilization, critical path delay, maximum operating frequency, and power consumption should be analyzed after synthesis. This would provide a more complete understanding of the processor not only as a functional Verilog model, but also as a hardware design that could be implemented on an FPGA or similar digital platform.

Overall, this assignment helped us understand the fundamental principles of processor design, especially the interaction between datapath modules and control logic. It also provided practical experience in using Verilog HDL, designing a finite state machine controller, integrating multiple hardware modules, and verifying a digital system through simulation.

Member Contribution*

No.	Member	Student ID	Contribution
1	Trần Vinh Quang	2550342	Implemented the Controller module, including the finite state machine and control signal generation. Contributed to the project report.
2	Lê Hiễn Vinh	2453422	Implemented the Program Counter and top-level CPU modules. Developed expected output files for testbenches, and performed simulation verification for the system.
3	Nguyễn Đức Thiên Tân	2550345	Implemented the Memory and Instruction Register modules, including memory access logic and instruction decoding. Contributed to the project report.
4	Đặng Ngọc Lan Anh	2550258	Implemented the ALU and Accumulator modules, including arithmetic, logic, zero flag, and register operations. Contributed to the project report.